

■ Angular + Azure APIM

Guide complet d'intégration avec Azure AD (OAuth2)

Angular 17+

MSAL v3

Azure APIM

Entra ID

OAuth2 / JWT

Ce guide vous accompagne étape par étape pour construire
une application Angular sécurisée appelant une API Azure APIM
protégée par Azure Active Directory / Entra ID.

Table des Matières

- 1.** Prérequis & Architecture
- 2.** Configuration Azure AD / Entra ID
- 3.** Création du projet Angular
- 4.** Intégration MSAL (Authentification)
- 5.** Configuration Azure API Management
- 6.** Service Angular → Appel APIM
- 7.** Protection des routes
- 8.** Variables d'environnement
- 9.** Build & Déploiement
- 10.** Sécurité & Bonnes pratiques
- 11.** Dépannage (Troubleshooting)

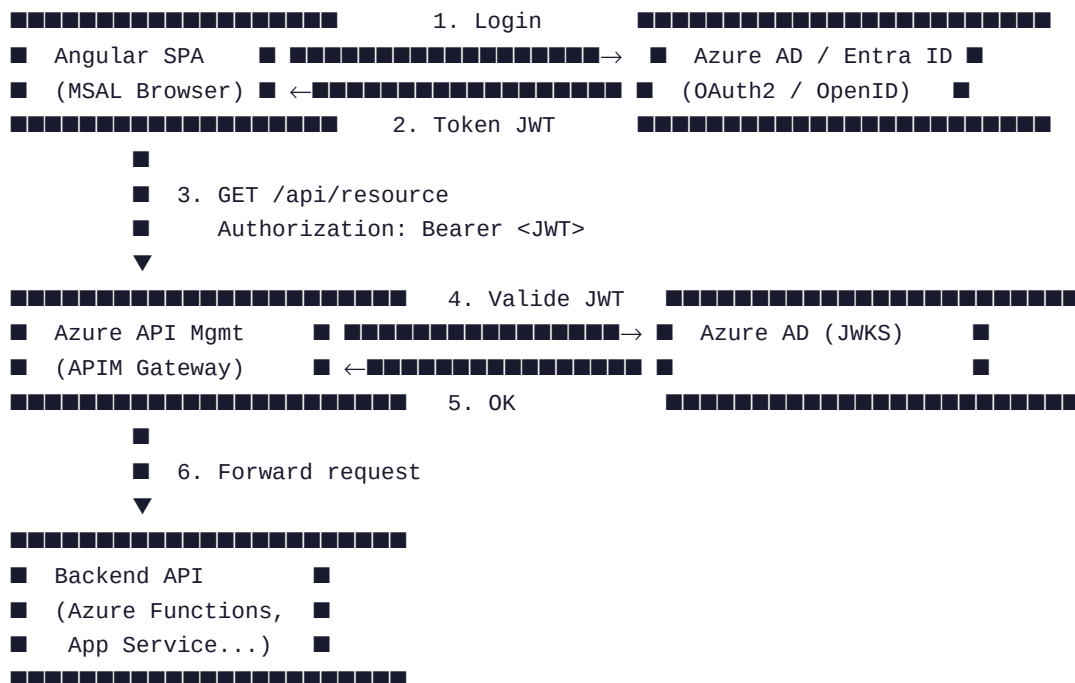
1 Prérequis & Architecture

Outils nécessaires

- ✓ Node.js 18.17+ — **node -v**
- ✓ Angular CLI 17+ — **npm install -g @angular/cli**
- ✓ Un abonnement Azure actif avec droits sur Entra ID et APIM
- ✓ Azure CLI (optionnel) — **az --version**

Architecture globale

■ Flux d'authentification : Angular (SPA) → Azure AD (token JWT) → APIM (validation JWT) → Backend API



2

Configuration Azure AD / Entra ID

2.1 — Enregistrer l'application Angular (SPA)

Dans le portail Azure → **Microsoft Entra ID** → **App registrations** → **+ New registration**

Champ	Valeur
Name	mon-app-angular
Supported account types	Accounts in this organizational directory only
Redirect URI (type)	Single-page application (SPA)
Redirect URI (URL)	http://localhost:4200

Notez ces valeurs — elles seront utilisées dans Angular :

Application (client) ID : xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx ← `clientId`
Directory (tenant) ID : yyyyyyyy-yyyy-yyyy-yyyy-yyyyyyyyyyyyy ← `tenantId`

2.2 — Exposer une API (scopes)

Dans l'App Registration → **Expose an API** :

Application ID URI → `api://<clientId>`
Scope name → `access_as_user`
Who can consent → Admins and users
State → Enabled

■ ■ Notez le scope complet : `api:///access_as_user`

2.3 — Enregistrer l'application Backend / API

Si votre backend a sa propre App Registration :

App Registration Backend → `mon-api-backend`
Expose API scope → `api://<backendClientId>/api.read`

Autorisez l'app Angular à accéder au backend : App Angular → **API permissions** → **Add permission** → Sélectionner l'API backend → cocher **access_as_user** → **Grant admin consent**.

3

Création du projet Angular

3.1 — Initialisation

```
# Créer le projet
ng new mon-app --routing --style=scss --standalone=false
cd mon-app

# Installer MSAL Angular v3
npm install @azure/msal-angular @azure/msal-browser

# Vérifier les versions
npx ng version
```

3.2 — Structure recommandée du projet

```
src/
  app/
    auth/
      auth-config.ts      ← Configuration MSAL
      auth.guard.ts       ← Guard de route
    core/
      services/
        api.service.ts    ← Appels APIM
        auth.service.ts   ← Service authentication
      interceptors/
        auth.interceptor.ts ← Ajout token aux requêtes
    features/
      home/
        home.component.ts
      dashboard/
        dashboard.component.ts
    app.module.ts
    app-routing.module.ts
  environments/
    environment.ts        ← Config dev
    environment.prod.ts   ← Config production
```

4 Intégration MSAL (Authentification)

4.1 — Fichier de configuration MSAL

src/app/auth/auth-config.ts

```
import { Configuration, BrowserCacheLocation, LogLevel } from '@azure/msal-browser';

export const msalConfig: Configuration = {
  auth: {
    clientId: 'VOTRE_CLIENT_ID',
    authority: 'https://login.microsoftonline.com/VOTRE_TENANT_ID',
    redirectUri: window.location.origin, // Auto-détecte l'URL
    postLogoutRedirectUri: window.location.origin,
  },
  cache: {
    cacheLocation: BrowserCacheLocation.LocalStorage,
    storeAuthStateInCookie: false, // true si IE11/Edge legacy
  },
  system: {
    loggerOptions: {
      loggerCallback: (level, message, containsPii) => {
        if (!containsPii) console.log(message);
      },
      logLevel: LogLevel.Warning,
    },
  },
};

// Scopes pour accéder à votre APIM
export const apiConfig = {
  scopes: ['api://VOTRE_CLIENT_ID/access_as_user'],
  uri: 'https://VOTRE_APIM.azure-api.net',
};

// Scopes Microsoft Graph (si besoin profil utilisateur)
export const graphConfig = {
  scopes: ['User.Read'],
  uri: 'https://graph.microsoft.com/v1.0/me',
};
```

4.2 — Module principal (app.module.ts)

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HTTP_INTERCEPTORS, HttpClientModule } from '@angular/common/http';
import {
  MsalModule, MsalInterceptor, MsalGuard,
  MsalRedirectComponent, MsalBroadcastService
} from '@azure/msal-angular';
import {
  PublicClientApplication, InteractionType, BrowserCacheLocation
} from '@azure/msal-browser';
import { msalConfig, apiConfig } from '../auth/auth-config';
import { AppRoutingModuleModule } from '../app-routing.module';
import { AppComponent } from '../app.component';
```

```
@NgModule({
  declarations: [AppComponent],
  imports: [
    BrowserModule,
    HttpClientModule,
    AppRoutingModule,
    MsalModule.forRoot(
      new PublicClientApplication(msalConfig),
      // Guard config
      {
        interactionType: InteractionType.Redirect,
        authRequest: { scopes: apiConfig.scopes },
      },
      // Interceptor config – map URL → scopes
      {
        interactionType: InteractionType.Redirect,
        protectedResourceMap: new Map([
          [apiConfig.uri + '/*', apiConfig.scopes],
        ]),
      }
    ),
  ],
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: MsalInterceptor,    // Ajoute automatiquement Bearer token
      multi: true,
    },
    MsalGuard,
    MsalBroadcastService,
  ],
  bootstrap: [AppComponent, MsalRedirectComponent],
})
export class AppModule {}
```

4.3 — Composant racine (app.component.ts)

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { MsalService, MsalBroadcastService } from '@azure/msal-angular';
import {
  EventMessage, EventType, InteractionStatus
} from '@azure/msal-browser';
import { Subject } from 'rxjs';
import { filter, takeUntil } from 'rxjs/operators';

@Component({
  selector: 'app-root',
  template: `
    <app-nav></app-nav>
    <main>
      <app-msal-redirect></app-msal-redirect>
      <router-outlet></router-outlet>
    </main>
  `
})
export class AppComponent implements OnInit, OnDestroy {
  private destroy$ = new Subject<void>();
  isAuthenticated = false;

  constructor(
    private msalService: MsalService,
    private broadcastService: MsalBroadcastService
  ) {}

  ngOnInit(): void {
    // Gérer le retour après redirect login
    this.msalService.handleRedirectObservable().subscribe();

    // Suivre l'état d'authentification
    this.broadcastService.inProgress$
      .pipe(
        filter(s => s === InteractionStatus.None),
        takeUntil(this.destroy$)
      )
      .subscribe(() => {
        this.isAuthenticated =
          this.msalService.instance.getAllAccounts().length > 0;
      });
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }

  login(): void {
    this.msalService.loginRedirect();
  }

  logout(): void {
    this.msalService.logoutRedirect();
  }
}
```



```
}  
}
```

5

Configuration Azure API Management

5.1 — Créer l'instance APIM

```
# Via Azure CLI
az apim create \
  --name mon-apim \
  --resource-group mon-rg \
  --publisher-email admin@company.com \
  --publisher-name "Mon Entreprise" \
  --sku-name Developer      # Developer pour dev/test, Standard/Premium pour prod
```

5.2 — Configurer CORS dans APIM

Dans APIM → **APIs** → votre API → **Inbound processing** → + **Add policy** → CORS :

```
<cors allow-credentials="true">
  <allowed-origins>
    <origin>http://localhost:4200</origin>
    <origin>https://votre-app.azurestaticapps.net</origin>
  </allowed-origins>
  <allowed-methods preflight-result-max-age="300">
    <method>GET</method>
    <method>POST</method>
    <method>PUT</method>
    <method>DELETE</method>
    <method>OPTIONS</method>
  </allowed-methods>
  <allowed-headers>
    <header>Authorization</header>
    <header>Content-Type</header>
    <header>Accept</header>
  </allowed-headers>
  <expose-headers>
    <header>*</header>
  </expose-headers>
</cors>
```

5.3 — Valider le JWT Azure AD dans APIM

Politique **validate-jwt** — Inbound processing :

```
<validate-jwt header-name="Authorization"
  failed-validation-httpcode="401"
  failed-validation-error-message="Token invalide ou expiré"
  require-expiration-time="true"
  require-signed-tokens="true">

  <openid-config url="https://login.microsoftonline.com/VOTRE_TENANT_ID/v2.0
    /.well-known/openid-configuration" />

  <audiences>
    <audience>api://VOTRE_CLIENT_ID</audience>
  </audiences>

  <issuers>
```

```
<issuer>https://sts.windows.net/VOTRE_TENANT_ID/</issuer>
<issuer>https://login.microsoftonline.com/VOTRE_TENANT_ID/v2.0</issuer>
</issuers>

<required-claims>
  <claim name="scp" match="any">
    <value>access_as_user</value>
  </claim>
</required-claims>

</validate-jwt>
```

■ Avec cette politique, APIM rejette automatiquement toute requête sans token valide (401 Unauthorized) — le backend n'est jamais appelé.

5.4 — Politique de rate limiting (optionnel)

```
<!-- Limiter à 10 appels par seconde par subscription -->
<rate-limit calls="10" renewal-period="1" />

<!-- Ou par IP -->
<rate-limit-by-key calls="100"
                  renewal-period="60"
                  counter-key="@context.Request.IpAddress)" />
```

5.5 — Politique complète recommandée

```
<policies>
  <inbound>
    <base />
    <!-- 1. CORS -->
    <cors allow-credentials="true">
      <allowed-origins>
        <origin>http://localhost:4200</origin>
        <origin>https://votre-app.azurestaticapps.net</origin>
      </allowed-origins>
      <allowed-methods><method>*</method></allowed-methods>
      <allowed-headers><header>*</header></allowed-headers>
    </cors>
    <!-- 2. Validation JWT -->
    <validate-jwt header-name="Authorization"
                  failed-validation-httpcode="401"
                  failed-validation-error-message="Non autorisé">
      <openid-config url="https://login.microsoftonline.com/
                       TENANT_ID/v2.0/.well-known/openid-configuration" />
      <audiences><audience>api://CLIENT_ID</audience></audiences>
    </validate-jwt>
    <!-- 3. Rate limiting -->
    <rate-limit calls="100" renewal-period="60" />
  </inbound>
  <backend><base /></backend>
  <outbound>
    <!-- Supprimer les headers internes -->
    <set-header name="X-Powered-By" exists-action="delete" />
    <base />
  </outbound>
  <on-error><base /></on-error>
</policies>
```

6

Service Angular → Appel APIM

6.1 — Service HTTP principal

src/app/core/services/api.service.ts

```

import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError, retry } from 'rxjs/operators';
import { environment } from '../../../environments/environment';

export interface ApiResponse<T> {
  data: T;
  status: number;
  message?: string;
}

@Injectable({ providedIn: 'root' })
export class ApiService {
  private baseUrl = environment.apimBaseUrl;

  constructor(private http: HttpClient) {}

  // GET – le token JWT est ajouté automatiquement par MsalInterceptor
  get<T>(endpoint: string): Observable<T> {
    return this.http
      .get<T>(`${this.baseUrl}/${endpoint}`)
      .pipe(
        retry(1),
        catchError(this.handleError)
      );
  }

  post<T>(endpoint: string, body: unknown): Observable<T> {
    return this.http
      .post<T>(`${this.baseUrl}/${endpoint}`, body)
      .pipe(catchError(this.handleError));
  }

  put<T>(endpoint: string, body: unknown): Observable<T> {
    return this.http
      .put<T>(`${this.baseUrl}/${endpoint}`, body)
      .pipe(catchError(this.handleError));
  }

  delete<T>(endpoint: string): Observable<T> {
    return this.http
      .delete<T>(`${this.baseUrl}/${endpoint}`)
      .pipe(catchError(this.handleError));
  }

  private handleError(error: any): Observable<never> {
    let errorMessage = 'Une erreur est survenue';
    if (error.status === 401) errorMessage = 'Non autorisé – Token expiré ?';
  }

```

```

    if (error.status === 403) errorMessage = 'Accès interdit';
    if (error.status === 0) errorMessage = 'Impossible de joindre l'API';
    console.error('API Error:', error);
    return throwError(() => new Error(errorMessage));
  }
}

```

6.2 — Utilisation dans un composant

```

import { Component, OnInit } from '@angular/core';
import { ApiService } from '../../core/services/api.service';

interface Product {
  id: number;
  name: string;
  price: number;
}

@Component({
  selector: 'app-products',
  template: `
    <div *ngIf="loading">Chargement...</div>
    <div *ngIf="error" class="error">{{ error }}</div>
    <ul *ngFor="let p of products">
      <li>{{ p.name }} – {{ p.price | currency:'EUR' }}</li>
    </ul>
  `,
})
export class ProductsComponent implements OnInit {
  products: Product[] = [];
  loading = false;
  error = '';

  constructor(private api: ApiService) {}

  ngOnInit(): void {
    this.loading = true;
    this.api.get<Product[]>('products').subscribe({
      next: (data) => {
        this.products = data;
        this.loading = false;
      },
      error: (err) => {
        this.error = err.message;
        this.loading = false;
      }
    });
  }
}

```

7 Protection des routes

src/app/app-routing.module.ts

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { MsalGuard } from '@azure/msal-angular';
import { BrowserUtils } from '@azure/msal-browser';

const routes: Routes = [
  {
    path: '',
    redirectTo: 'dashboard',
    pathMatch: 'full'
  },
  {
    path: 'dashboard',
    loadChildren: () =>
      import('./features/dashboard/dashboard.module')
        .then(m => m.DashboardModule),
    canActivate: [MsalGuard], // ← Protégé par Azure AD
  },
  {
    path: 'products',
    loadChildren: () =>
      import('./features/products/products.module')
        .then(m => m.ProductsModule),
    canActivate: [MsalGuard],
  },
  {
    path: 'login-failed', // Page d'erreur auth
    component: LoginFailedComponent,
  },
];

@NgModule({
  imports: [RouterModule.forRoot(routes, {
    // Ne pas effectuer initial navigation si dans un iframe (MSAL)
    initialNavigation:
      !BrowserUtils.isInIframe() && !BrowserUtils.isInPopup()
      ? 'enabledBlocking'
      : 'disabled',
  })],
  exports: [RouterModule],
})
export class AppRoutingModule {}
```

8

Variables d'environnement

src/environments/environment.ts (développement)

```
export const environment = {  
  production: false,  
  clientId:    'VOTRE_CLIENT_ID',  
  tenantId:    'VOTRE_TENANT_ID',  
  apimBaseUrl: 'https://VOTRE_APIM.azure-api.net',  
  scopes:      ['api://VOTRE_CLIENT_ID/access_as_user'],  
  redirectUri: 'http://localhost:4200',  
};
```

src/environments/environment.prod.ts (production)

```
export const environment = {  
  production: true,  
  clientId:    'VOTRE_CLIENT_ID',           // Même app registration  
  tenantId:    'VOTRE_TENANT_ID',  
  apimBaseUrl: 'https://VOTRE_APIM.azure-api.net',  
  scopes:      ['api://VOTRE_CLIENT_ID/access_as_user'],  
  redirectUri: 'https://votre-app.azurestaticapps.net',  
};
```

■ Ne jamais commiter les IDs dans un dépôt public. Utilisez des variables d'environnement CI/CD ou Azure Key Vault pour injecter ces valeurs lors du build.

Injecter via Angular avec file replacements dans angular.json :

```
"fileReplacements": [  
  {  
    "replace": "src/environments/environment.ts",  
    "with": "src/environments/environment.prod.ts"  
  }  
]
```


9 Build & Déploiement

9.1 — Build de production

```
# Build optimisé
ng build --configuration=production

# Sortie dans : dist/mon-app/
```

9.2 — Déploiement sur Azure Static Web Apps (recommandé)

```
# Installer l'extension Azure Static Web Apps CLI
npm install -g @azure/static-web-apps-cli

# Créer la ressource Azure (via CLI)
az staticwebapp create \
  --name mon-app-angular \
  --resource-group mon-rg \
  --source https://github.com/user/repo \
  --location "West Europe" \
  --branch main \
  --app-location "/" \
  --output-location "dist/mon-app"
```

9.3 — Configuration staticwebapp.config.json

Nécessaire pour que le routing Angular fonctionne :

```
{
  "routes": [
    {
      "route": "/*",
      "serve": "/index.html",
      "statusCode": 200
    }
  ],
  "navigationFallback": {
    "rewrite": "/index.html",
    "exclude": ["/assets/*", "*.css,js,png,jpg,svg"]
  },
  "mimeTypes": {
    ".json": "application/json"
  }
}
```

9.4 — Ajouter l'URL de production dans Azure AD

Dans l'App Registration Angular → **Authentication** → **+ Add URI** → ajouter <https://votre-app.azurestaticapps.net>

10

Sécurité & Bonnes pratiques

- ✓ **PKCE activé par défaut** avec MSAL v3 pour les SPA — ne pas utiliser le flux implicite.
- ✓ **Ne jamais stocker le token dans sessionStorage/localStorage manuellement** — laisser MSAL gérer le cache.
- ✓ **Rotation des secrets** : pour les app registrations backend, utiliser des certificats plutôt que des secrets clients.
- ✓ **Durée de vie des tokens** : access token 1h, refresh token 24h par défaut — configurable dans Entra ID.
- ✓ **Révoquer les tokens** en cas de compromission via la Conditional Access Policy d'Entra ID.
- ✓ **APIM Subscription Key** : même avec JWT, ajoutez une subscription key comme couche de défense additionnelle.
- ✓ **Logs APIM** : activer Azure Monitor / Application Insights pour tracer tous les appels.
- ✓ **Content Security Policy** : ajouter des headers CSP pour protéger l'Angular SPA contre les XSS.

Header CSP recommandé (via APIM outbound policy) :

```
<set-header name="Content-Security-Policy" exists-action="override">
  <value>
    default-src 'self';
    script-src 'self';
    style-src 'self' 'unsafe-inline';
    connect-src 'self' https://login.microsoftonline.com
                https://VOTRE_APIM.azure-api.net;
    img-src 'self' data;;
    frame-ancestors 'none';
  </value>
</set-header>
```

11

Dépannage (Troubleshooting)

■ AADSTS50011 — The redirect URI does not match

L'URL dans l'App Registration ne correspond pas à `redirectUri`.

→ Vérifier que `http://localhost:4200` est bien enregistré dans Authentication → Redirect URIs (type SP).

■ 401 Unauthorized depuis APIM

Le token JWT n'est pas envoyé ou est invalide.

→ Vérifier que `MsalInterceptor` est bien enregistré dans `providers`.

→ Vérifier la `protectedResourceMap` : l'URL doit matcher exactement.

■ CORS Error sur l'appel APIM

La politique CORS APIM ne couvre pas votre origine.

→ Ajouter l'URL dans `<allowed-origins>` dans la politique APIM.

→ La politique CORS doit être placée AVANT `validate-jwt` dans `inbound`.

■ Token expiré — boucle de refresh infinie

Le refresh token a expiré ou a été révoqué.

→ Implémenter un handler d'erreur 401 qui appelle `msalService.loginRedirect()`.

■ `interaction_in_progress` — erreur MSAL

Une interaction de login est déjà en cours (double appel).

→ S'assurer que `handleRedirectObservable()` est appelé dans `AppComponent.ngOnInit()`.

→ Utiliser le filtre `InteractionStatus.None` avant d'initier un login.

■ APIM retourne 404 Not Found

Le chemin de l'API dans APIM ne correspond pas à l'URL appelée.

→ Vérifier le suffixe de l'API et les opérations dans le portail APIM.

Récapitulatif — Checklist de déploiement

- ✓ App Registration Angular créée (type SPA, redirect URI configurée)
- ✓ Scope api:///access_as_user exposé et consenti
- ✓ msalConfig configuré avec clientId et tenantId corrects
- ✓ MsalModule.forRoot() configuré dans AppModule
- ✓ protectedResourceMap mappant l'URL APIM aux scopes
- ✓ MsalInterceptor enregistré dans providers
- ✓ handleRedirectObservable() appelé dans AppComponent
- ✓ Routes protégées avec canActivate: [MsalGuard]
- ✓ Politique CORS dans APIM (localhost + domaine prod)
- ✓ Politique validate-jwt dans APIM avec openid-config correct
- ✓ URL de production ajoutée dans l'App Registration
- ✓ staticwebapp.config.json configuré pour le routing SPA
- ✓ Variables d'environnement prod injectées via CI/CD

■ Ressources utiles : • MSAL Angular : <https://github.com/AzureAD/microsoft-authentication-library-for-js> • Azure APIM Policies : <https://learn.microsoft.com/azure/api-management/api-management-policies> • Entra ID App Registration : <https://learn.microsoft.com/azure/active-directory/develop/quickstart-register-app>